CombatSim Behavior Node System Documentation

This document serves as the complete technical and design reference for the **Behavior Tree & State Machine System** in CombatSim. It details how the engine models creature artificial intelligence, executes state machines, advances directed node graphs, and calculates kinematics, swarm dynamics, and combat damage for every node in the simulator.

Table of Contents

1. System Architecture & Runtime Execution Model
    - State Machine + Behavior Graph Hybrid
    - Per-Tick Update Pipeline (`tickCreatureBehavior`)
    - Node Connection Handles
    - Entity Dynamics: Single Boss vs Swarm / Boids
2. Action Nodes (Blue)
    - `approach`
    - `deal_damage`
    - `retreat`
    - `run_away`
    - `hide`
    - `wait_idle`
    - `scatter`
    - `reform`
    - `boids_flock`
    - `split_swarm_groups`
    - `group_approach_building`
    - `group_strike_target`
    - `return_to_swarm`
    - `charge_orbit`
    - `short_circuit_drain`
    - `flock_dissipate_regroup`

3. Condition Nodes (Yellow)
    - `until_dealt_damage`
    - `until_received_damage`
    - `until_aggression`
    - `for_seconds`
    - `once`
    - `health_threshold`
    - `distance_check`
    - `swarm_size`
    - `formation_integrity`
4. Flow Control Nodes (Green)
    - `repeat`
    - `branch_switch`
    - `state_transition`
    - `parallel_execute`
5. Special Nodes (Purple)
    - `swarm_coordination`
    - `phase_trigger`
    - `event_listener`
6. Swarm Kinematics & Flocking Mechanics
7. Apex Predator Specialized Mechanics
8. Edge Cases & Safety Fallbacks

1. System Architecture & Runtime Execution Model

State Machine + Behavior Graph Hybrid

CombatSim unifies **Hierarchical State Machines (HSM)** and **Directed Graph Behavior Trees**:

- **States (`BehaviorState`)**: Represent macro phases of combat (e.g. *Stalking*, *Cruising Orbit*, *Deck Assault*, *Enraged*). Each state owns an entry node (`entryNodeId`), priority, and a collection of graph nodes.

- **Nodes (`BehaviorNode`)**: Fine-grained tactical actions, conditional gates, or flow routers. Nodes belong to a specific `stateId`.
- **Connections (`BehaviorConnection`)**: Directed links transferring control between nodes on success (`next`), true evaluations (`true`), false evaluations (`false`), or concurrent execution (`parallel`).

Per-Tick Update Pipeline (`tickCreatureBehavior`)

Every simulation tick ($dt \approx 0.016$sdt ≈ 0.016s at 60 FPS), the runner executes:

1. **Timer Accumulation**: Increments node active timer (`nodeTimer += dt`).
2. **Submersion Check**: If active node is **not** `hide` or deep dive in `charge_orbit`, depth ratio decays toward 0 (surfacing), leaving the entity vulnerable to weapon fire.
3. **Spatial Metrics**: Calculates distance (dd) and heading angle (θθ) from the creature to the Behemoth vessel center (x_B, y_B)(xB, yB).
4. **Node Logic Execution**: Evaluates the active node's physics, attack cooldowns, or branching checks.
5. **Event Emission**: Generates combat events (`STRIKE_HULL`, `SHOOT_DECK`, `RAM_IMPACT`, `SHORT_CIRCUIT_DRAIN`, `GROUP_STRIKE`, `SURFACED`, `SUBMERGED`).
6. **Centroid & Position Update**: Integrates linear velocities (v_x, v_y)(vx, vy) for single creatures, or computes the center of mass (centroid) across all alive swarm entities.

Node Connection Handles

Handle Name	Used By	Semantic Meaning
`next`	Action, Special, Flow	Standard unconditional forward sequence on completion.
`true`	Condition, Flow (`repeat`)	Branch taken when condition test succeeds or loop iteration continues.
`false`	Condition, Flow (`repeat`)	Branch taken when condition test fails or loop iteration exhausts count.
`parallel`	`parallel_execute`	Triggers concurrent execution branches simultaneously.

Entity Dynamics: Single Boss vs Swarm / Boids

- **Single Creatures** (e.g. *Thunder Leviarch*, *Aetherfin*): Governed by primary position coordinates (x, y)(x, y), heading angle θθ, and translational velocities (v_x, v_y)(vx, vy).
- **Swarm Entities** (e.g. *Veilfangs*): Possess an array of individual autonomous entities (`swarmEntities`), each with its own coordinates (x_i, y_i)(xi, yi), velocity $(v_{x,i}, v_{y,i})$(vx,i, vy,i), individual HP, and squad assignments. The parent creature coordinates (x, y)(x, y) represent the dynamic centroid of living swarm members.

2. Action Nodes (Blue)

Action nodes perform active movement, offensive strikes, evasive maneuvers, or positioning logic.

`approach`

Moves the entity directly toward the Behemoth until reaching a minimum engagement distance.

- **Category**: `action`
- **Icon / Visual**: `Move` (Blue)
- **Parameters**:
  - `speed` (number, knots): Speed of approach. Defaults to creature `movementSpeed`.
  - `minDistance` (number, px): Stopping distance threshold from vessel center. Default `45px`.
  - `nextNodeId` (string): Node to execute upon reaching `minDistance`.
- **Runtime Logic**: $\theta = \text{atan2}(y_B - y, x_B - x)$θ = atan2(yB − y, xB − x) $v_x = \cos(\theta) \cdot \text{speed}, \quad v_y = \sin(\theta) \cdot \text{speed}$vx = cos(θ) · speed, vy = sin(θ) · speed When $d \le \text{minDistance}$d ≤ minDistance, velocities are damped ($v \leftarrow v \cdot 0.5$v ← v · 0.5) and `advanceTo(nextNodeId)` triggers.
- **Combat Events**: None directly; positions creature for subsequent melee or ramming.

`deal_damage`

Executes an attack strike against the ship hull, deck installations, or vessel modules.

- **Category**: `action`
- **Icon / Visual**: `Flame` (Blue)
- **Parameters**:

- `damage` (number, HP): Strike damage magnitude. Defaults to creature `strikeDamageRate`.
- `cooldown` (number, sec): Execution delay before triggering attack. Default `2.0s`.
- `damageTarget` (`'hull' | 'buildings' | 'crew'`): Target subsystem. Falls back to creature `attackTarget`.
- `damageType` (`'kinetic' | 'energy' | 'explosive'`): Damage category for armor absorption.
- `nextNodeId` (string): Node triggered following attack delivery.

- **Runtime Logic**: Waits until `nodeTimer >= cooldown`. Emits either `SHOOT_DECK` (if `canShootDeck` is true) or `RAM_IMPACT`. Increments `dealtDamageCounter` by `damage`, logs event, and calls `advanceTo(nextNodeId)`.
- **Combat Events Emitted**:
  - `RAM_IMPACT` or `SHOOT_DECK` with `damage`, `stunChance`, `stunDuration`, `ramMass`, and `strikeSide` (`port` if $y < y_B$ y < yB, else `starboard`).

`retreat`

Tactically backs away from the Behemoth to create distance before re-engaging.

- **Category**: `action`
- **Icon / Visual**: `Compass` (Blue)
- **Parameters**:
  - `speed` (number, knots): Disengagement speed. Default `movementSpeed`.
  - `minDistance` (number, px): Safety distance threshold. Default `350px`.
  - `nextNodeId` (string): Next node once safe distance is attained.
- **Runtime Logic**: Moves in exact opposite direction of vessel: $\theta_{\text{away}} = \theta_{\text{Behemoth}} + 180^\circ$ θaway = θBehemoth + 180∘ $v_x = \cos(\theta_{\text{away}}) \cdot \text{speed}, \quad v_y = \sin(\theta_{\text{away}}) \cdot \text{speed}$vx = cos(θaway) ⋅ speed, vy = sin(θaway) ⋅ speed When $d \geq \text{minDistance}$d ≥ minDistance, velocity is damped and `advanceTo(nextNodeId)` triggers.

`run_away`

High-speed escape acceleration away from the combat zone (aggro break).

- **Category**: `action`
- **Icon / Visual**: `ArrowRight` (Blue)

- **Parameters**:
  - `speed` (number, knots): Escape velocity. Defaults to $1.5 \times \text{movementSpeed}$ 1.5 × movementSpeed.
  - `nextNodeId` (string): Node executed once escaped beyond 750px750px.
- **Runtime Logic**: Accelerates directly away from Behemoth center. When $d > 750\text{px}$d > 750px, triggers `advanceTo(nextNodeId, 'Escaped combat zone')`.

`hide`

Submerges the creature underwater, applying full damage immunity.

- **Category**: `action`
- **Icon / Visual**: `EyeOff` (Blue)
- **Parameters**:
  - `duration` (number, sec): Immersion time window. Default `underwaterInterval`.
  - `nextNodeId` (string): Next node executed upon surfacing.
- **Runtime Logic**: Sets `creature.underwater = true`. Interpolates `depthRatio` to $1.0$1.0 at rate $2.0 \cdot dt$2.0 · dt. While submerged, weapon projectiles and deck fire cannot damage the creature. When `nodeTimer >= duration`, sets `underwater = false`, emits `SURFACED` event, and advances.

`wait_idle`

Halts active navigation, holding position for a set time window.

- **Category**: `action`
- **Icon / Visual**: `Clock` (Blue)
- **Parameters**:
  - `duration` (number, sec): Wait duration. Default `2.0s`.
  - `nextNodeId` (string): Next node executed when timer expires.
- **Runtime Logic**: Applies friction damping: $v_x \leftarrow v_x \cdot 0.85$vx ← vx · 0.85, $v_y \leftarrow v_y \cdot 0.85$vy ← vy · 0.85. Advances when `nodeTimer >= duration`.

`scatter`

Forces swarm entities to break formation and disperse in random chaotic directions.

- **Category**: `action`
- **Icon / Visual**: `Sparkles` (Blue)
- **Parameters**:
  - `spreadRadius` (number, px): Dispersion boundary radius. Default `120px`.
  - `scatterSpeed` (number, knots): Speed of dispersal. Default $1.2 \times \text{movementSpeed}$ 1.2 × movementSpeed.
  - `duration` (number, sec): Duration of chaotic scattering. Default `2.5s`.
  - `nextNodeId` (string): Triggered when duration elapses.
- **Runtime Logic**: Decreases `formationIntegrity` at $45\%/\text{sec}$45%/sec down to a minimum of 15%15%. Generates randomized direction vectors with sinusoidal fluttering $\sin(\text{simTime} \cdot 12 + i \cdot 2.3) \cdot 0.45$sin(simTime · 12 + i · 2.3) · 0.45 and per-entity speed variation ($0.75 - 1.25 \times$ 0.75 – 1.25×). Advances when `nodeTimer >= duration`.

`reform`

Rallies dispersed swarm entities back into an orderly geometric formation.

- **Category**: `action`
- **Icon / Visual**: `Users` (Blue)
- **Parameters**:
  - `formationType` (`'wedge' | 'circle' | 'line' | 'grid'`): Shape template.
  - `reformSpeed` (number, knots): Convergence speed. Default `movementSpeed`.
  - `spacing` (number, px): Spacing between swarm slots. Default `20px`.
  - `nextNodeId` (string): Node executed when formation is restored.
- **Runtime Logic**: Increases `formationIntegrity` at $35\%/\text{sec}$35%/sec up to 100%100%. Calculates target formation offsets per index:
  - **Circle**: $r = \max(20, \sqrt{N} \cdot \text{spacing} \cdot 0.7)$r = max(20, N

· spacing · 0.7), $\alpha = i \cdot \frac{360^\circ}{N}$α = i · N360∘.
  - **Wedge (V-formation)**: Rank step $-(\text{rank} \cdot \text{spacing}) \cdot \cos(145^\circ)$– (rank · spacing) · cos(145∘), lateral side $\pm 1 \cdot (\text{rank} \cdot \text{spacing} \cdot 0.9)$ ±1 · (rank · spacing · 0.9).
  - **Line**: Rank step $0$0, lateral offset $\pm 1 \cdot \text{rank} \cdot \text{spacing}$±1 · rank · spacing.
  - **Grid**: Square bounding columns $\lceil \sqrt{N} \rceil$⌈ N

] with row/column centering. Smoothly lerps each entity to its designated formation slot. Advances when `formationIntegrity >= 85%` or `nodeTimer >= 3.5s`.

`boids_flock`

Autonomous flocking behavior simulating natural flock dynamics (Craig Reynolds model) while orbiting the Behemoth.

- **Category**: `action`
- **Icon / Visual**: `Users` (Blue)
- **Parameters**:
  - `speed` (number, knots): Cruising flock velocity. Default `200 knots`.
  - `duration` (number, sec): Active cruising duration before routing. Default `3.5s`.
  - `flockSeparationDist` (number, px): Crowding avoidance threshold. Default `24px`.
  - `reactionCooldown` (number, sec): Cooldown between reactive scatter triggers. Default `2.0s`.
  - `onHitNodeId` / `trueNodeId` (string): Optional branch taken if a flock member is struck by turret fire.
  - `nextNodeId` (string): Normal branch taken once cruising duration finishes.
- **Runtime Logic**:
  1. **Reactive Scatter Gate**: Checks if `lastHitPoint` occurred within 0.35s0.35s and cooldown passed. If true, immediately diverts execution to `onHitNodeId` or `trueNodeId`.
  2. **Orbital Roaming Anchor**: Maintains a circular orbital reference point at radius 300px 300px from Behemoth center, advancing at angular velocity $\omega = \frac{\text{speed}}{300} \cdot \frac{180}{\pi} \cdot dt$ ω = 300speed · π180 · dt.
  3. **Three Reynolds Boids Forces**:
     - **Separation**: Repels entities within `flockSeparationDist`: $\vec{F}_{\text{sep}} = \sum_{j \neq i, d < d_{\text{sep}}} \frac{\vec{r}_i - \vec{r}_j}{d} \cdot (d_{\text{sep}} - d)$F⃗sep = ∑j≠i,d<dsep dr⃗i−r⃗j · (dsep − d)
     - **Alignment**: Matches average neighbor velocity within 120px120px: $\vec{F}_{\text{align}} = \frac{1}{K} \sum \vec{v}_j$ F⃗align = K1∑ v⃗j
     - **Cohesion**: Steers toward local center of mass of neighbors: $\vec{F}_{\text{coh}} = \frac{1}{K} \sum \vec{r}_j - \vec{r}_i$ F⃗ coh = K1∑ r⃗j − r⃗i
  4. **Combined Steering Vector**: $\vec{F}_{\text{steer}} = 0.75\vec{v}_{\text{tangent}} + 0.8\vec{r}_{\text{anchor}} + 2.5\vec{F}_{\text{sep}} + 0.35\vec{F}_{\text{align}} + 0.15\vec{F}_{\text{coh}}$ F⃗steer = 0.75v⃗tangent + 0.8r⃗anchor + 2.5F⃗sep + 0.35F⃗align + 0.15F⃗cohMaintains

full constant cruising speed without velocity decay. Advances when `nodeTimer >= duration`.

`split_swarm_groups`

Partitions surviving swarm entities into autonomous attack squads.

- **Category**: `action`
- **Icon / Visual**: `Sparkles` (Blue)
- **Parameters**:
  - `groupMinSize` (number): Minimum squad count (e.g. 3 entities).
  - `groupMaxSize` (number): Maximum squad count (e.g. 5 entities).
  - `nextNodeId` (string): Next node following group division.
- **Runtime Logic**: Iterates over alive entities and slices them into discrete chunks sized randomly between `groupMinSize` and `groupMaxSize`. Populates `runtimeState.swarmGroups` with squad IDs (`pack_1`, `pack_2`), assigned entity indices, and state `approaching`. Advances immediately.

`group_approach_building`

Directs formed squads toward specific Behemoth building slots on deck.

- **Category**: `action`
- **Icon / Visual**: `Move` (Blue)
- **Parameters**:
  - `speed` (number, knots): Ingress flight speed. Defaults to $1.35 \times \text{movementSpeed}$ 1.35 × movementSpeed.
  - `targetBuildingSlotId` (string, optional): Specific building target ID. If omitted, targets alive/manned buildings.
  - `nextNodeId` (string): Triggered when all squads arrive over their target installations.
- **Runtime Logic**: If squads do not exist, partitions entities automatically into squads of 4. Maps each squad to an active building slot (e.g. Bridge, Generator, Accumulator, Medical Bay) taking row offsets (`misc_a` vs `misc_b`). Steers squad members toward target deck coordinates. Advances when all entities arrive within 35px35px of targets or `nodeTimer >= 3.5s`.

`group_strike_target`

Conducts sustained dive-bombing assault against targeted building structures or assigned crew members.

- **Category**: `action`
- **Icon / Visual**: `Flame` (Blue)
- **Parameters**:
  - `duration` (number, sec): Assault duration. Default `2.0s`.
  - `crewAttackChance` (number, $0.0 - 1.0$0.0 – 1.0): Probability of targeting assigned crew directly instead of building HP. Default `0.60` (60%).
  - `damage` (number, HP): Total damage dealt across the squad. Default `45 HP`.
  - `nextNodeId` (string): Disengagement trigger.
- **Runtime Logic**: Entities perform frenzied swirling hover circles over the installation ($\text{radius} = 60\text{px}$radius = 60px). Every 0.8s0.8s, a strike tick executes:
  - If building has assigned crew and $\text{random}() < \text{crewAttackChance}$random() < crewAttackChance, deals direct lethal/injurious damage to a stationed crew member (`target: 'crew'`).
  - Otherwise, damages building structural HP (`target: 'buildings'`). Emits `GROUP_STRIKE` events and advances when `nodeTimer >= duration`.

`return_to_swarm`

Commands assault squads to break off deck engagement and fly outward to rejoin the main orbiting flock.

- **Category**: `action`
- **Icon / Visual**: `Compass` (Blue)
- **Parameters**:
  - `speed` (number, knots): Return flight speed. Defaults to $1.4 \times \text{movementSpeed}$ 1.4 × movementSpeed.
  - `nextNodeId` (string): Triggered once entities rejoin outer orbit.
- **Runtime Logic**: Sets staging waypoint at radius 300px300px in the current orbit sector. Directs all entities outward. Clears `swarmGroups` when all entities arrive within 50px50px or `nodeTimer >= 3.0s`, then advances.

`charge_orbit`

Specialized apex predator cruising maneuver (e.g. *Aetherfin Hunter*). Executes a continuous circular orbit while periodically surfacing and submerging.

- **Category**: `action`
- **Icon / Visual**: `Repeat` (Blue)
- **Parameters**:
  - `speed` (number, knots): Orbit velocity. Default `165 knots`.
  - `orbitRadius` (number, px): Ring radius from Behemoth center. Default `260px`.
  - `orbitDirection` (`'counter_clockwise' | 'clockwise'`): Orbit direction. Default `counter_clockwise`.
  - `surfaceInterval` (number, sec): Surfaced time (vulnerable). Default `3.0s`.
  - `depthsInterval` (number, sec): Submerged time (immune). Default `3.0s`.
  - `stageDuration` (number, sec): Total charging duration before attack transition. Default `18.0s`.
  - `nextNodeId` (string): Node executed when stage duration finishes.
- **Runtime Logic**: Smoothly integrates orbital angle αα: $\alpha \leftarrow \alpha \pm \frac{\text{speed}}{\text{radius}} \cdot \frac{180^\circ}{\pi} \cdot dt$ α ← α ± radiusspeed ⋅ π180∘ ⋅ dt If creature is far from the orbit ring ($> 35\text{px}$> 35px), intercepts smoothly; once on the ring, locks into a true tangent vector$\vec{v}_{\text{tangent}}$ v⃗tangent. Periodically alternates between deep dive (`underwater = true`, `depthRatio -> 1.0`) and surface (`underwater = false`, `depthRatio -> 0.0`). Advances when `nodeTimer >= stageDuration`.

`short_circuit_drain`

High-risk apex attack where the creature slides beneath the Behemoth hull, establishing direct contact to drain shipboard power systems.

- **Category**: `action`
- **Icon / Visual**: `Zap` (Blue)
- **Parameters**:
  - `duration` (number, sec): Hull contact duration. Default `10.0s`.
  - `stunChance` (number, $0.0 - 1.0$0.0 – 1.0): Probability per second to stun systems. Default `0.10`.
  - `stunDuration` (number, sec): Duration of system paralysis. Default `10.0s`.
  - `nextNodeId` (string): Node triggered when drain finishes.

- **Runtime Logic**: Approaches vessel center at $0.75 \times \text{speed}$0.75 × speed. Sets `underwater = true` and `depthRatio = 1.0`. Once within 45px45px, latches beneath hull (`drainArrived = true`). Emits continuous `SHORT_CIRCUIT_DRAIN` events to deplete vessel accumulator energy and stun modules. Advances when `nodeTimer >= duration`.

`flock_dissipate_regroup`

All-in-one reactive defense behavior for aerial swarms (*Veilfangs*). Triggered upon weapon impact, it executes a 3-phase sequence: explosive scatter, opportunistic squad assault on deck buildings, and regrouping at an opposite orbital sector.

- **Category**: `action`
- **Icon / Visual**: `Sparkles` (Cyan)
- **Parameters**:
  - `scatterSpeed` (number, knots): Dispersal velocity. Default `180 knots`.
  - `speed` (number, knots): Normal flight velocity. Default `200 knots`.
  - `dissipateDuration` (number, sec): Scatter burst time. Default `0.8s`.
  - `formSquadsBeforeRegroup` (boolean): Whether to assault buildings before reforming. Default `true`.
  - `squadAssaultDuration` (number, sec): Deck assault duration. Default `2.2s`.
  - `regroupDuration` (number, sec): Convergence duration. Default `2.2s`.
  - `regroupAngleOffset` (number, deg): Sector jump offset. Default `150°`.
  - `crewAttackChance` (number, $0.0 - 1.0$0.0 – 1.0): Crew strike probability. Default `0.60`.
  - `damage` (number, HP): Damage dealt per squad. Default `45 HP`.
  - `groupMinSize` / `groupMaxSize` (numbers): Squad size constraints (e.g. 3 to 5).
  - `nextNodeId` (string): Triggered when regrouping completes.
- **Runtime Logic (3 Phased Finite State Machine)**:
  1. **Phase 1: Dissipating (`scatterTimer < dissipateDuration`)**:
     - Epicenter set to weapon hit location.
     - Entities explode radially away with randomized tangential jitter ($\pm 65^\circ$ ±65∘) and lateral aerodynamic flutter ($\sin(\text{simTime} \cdot 14 + i \cdot 2.1) \cdot 75$ sin(simTime · 14 + i · 2.1) · 75).
     - Soft orbital envelope tether ($210\text{px} \le d \le 330\text{px}$210px ≤ d ≤ 330px) prevents swarms from scattering off-screen.
  2. **Phase 2: Squad Assault (`scatterTimer < squadAssaultDuration`)**:

        - Entities partition into squads targeting separate active building installations.
        - Ingresses onto deck and executes periodic strike ticks (damaging structures or crew members).
    3. **Phase 3: Regrouping (`scatterTimer < regroupDuration`)**:
        - Designates an orbital rendezvous point offset by $120^{\circ} - 180^{\circ}$ 120∘ – 180∘ on the opposite side of the ship.
        - Entities converge with swirling aerodynamic flutter into a unified flock.
    4. Resets to `idle` and calls `advanceTo(nextNodeId)`.

3. Condition Nodes (Yellow)

Condition nodes act as conditional decision branches. They test a simulation metric and route execution to either a `trueNodeId` or `falseNodeId`.

`until_dealt_damage`

Monitors accumulated damage inflicted by the creature on the Behemoth.

- **Category**: `condition`
- **Parameters**:
    - `targetDamage` (number, HP): Required damage threshold. Default `300 HP`.
    - `trueNodeId`: Taken when `dealtDamageCounter >= targetDamage`.
    - `falseNodeId`: Fallback taken on timeout ($> 10.0\text{s}$> 10.0s).
- **Logic**: When threshold is reached, resets `dealtDamageCounter = 0` and advances along `trueNodeId`.

`until_received_damage`

Detects incoming fire taken by the creature from defensive turrets.

- **Category**: `condition`
- **Parameters**:
    - `targetDamage` (number, HP): Damage received threshold. Default `150 HP`.
    - `trueNodeId`: Taken when `receivedDamageCounter >= targetDamage`.
    - `falseNodeId`: Fallback taken on timeout ($> 8.0\text{s}$> 8.0s).

- **Logic**: Triggered when turret hits accumulate `targetDamage`. Resets counter and branches to `trueNodeId`.

`until_aggression`

Tests the creature's emotional/tactical aggression level ($0 – 100\%$0 – 100%).

- **Category**: `condition`
- **Parameters**:
  - `targetAggression` (number, $0 – 100$0 – 100): Aggression threshold. Default `60`.
  - `aggressionOp` (`'<=' | '>=' | '==' | '<' | '>'`): Comparison operator. Default `>=`.
  - `trueNodeId` / `falseNodeId`: Target nodes.
- **Logic**: Evaluates `creature.aggression [op] targetAggression`. Routes immediately to `trueNodeId` if satisfied, else `falseNodeId`.

`for_seconds`

Pure time-window delay gate.

- **Category**: `condition`
- **Parameters**:
  - `duration` (number, sec): Time window. Default `3.0s`.
  - `trueNodeId`: Node executed once `nodeTimer >= duration`.
- **Logic**: Blocks until `duration` seconds elapse, then proceeds.

`once`

One-shot latch that fires its `true` branch on the first pass, and `false` on all subsequent passes.

- **Category**: `condition`
- **Parameters**:
  - `trueNodeId`: First pass target.
  - `falseNodeId`: Subsequent passes target.
- **Logic**: Checks `runtimeState.onceFlags[node.id]`. If false, sets flag to true and routes to `trueNodeId`. Otherwise routes to `falseNodeId`.

`health_threshold`

Tests creature remaining HP percentage.

- **Category**: `condition`
- **Parameters**:
  - `healthPercent` (number, $0 – 100\%$0 – 100%): Health threshold. Default `50%`.
  - `healthOp` (`'<=' | '>=' | '<'`): Comparison operator. Default `<=`.
  - `trueNodeId` / `falseNodeId`: Target nodes.
- **Logic**: $\text{currentHpPct} = \frac{\text{creature.hp}}{\text{creature.maxHp}} \cdot 100$currentHpPct = creature.maxHpcreature.hp ⋅ 100 If `currentHpPct [op] healthPercent` holds true, advances to `trueNodeId`, else `falseNodeId`.

`distance_check`

Tests physical distance from the creature centroid to the Behemoth vessel center.

- **Category**: `condition`
- **Parameters**:
  - `distanceThreshold` (number, px): Distance in pixels. Default `250px`.
  - `distanceOp` (`'<=' | '>=' | '<' | '>'`): Comparison operator. Default `<=`.
  - `trueNodeId` / `falseNodeId`: Target nodes.
- **Logic**: Computes $d = \text{hypot}(x_B - x, y_B - y)$d = hypot(xB – x, yB – y) and branches based on comparison operator.

`swarm_size`

Tests the count of surviving entities in a swarm.

- **Category**: `condition`
- **Parameters**:
  - `targetSwarmSize` (number): Entity count threshold. Default `5`.
  - `swarmSizeOp` (`'<=' | '>='`): Comparison operator. Default `<=`.
  - `trueNodeId` / `falseNodeId`: Target nodes.
- **Logic**: Filters `swarmEntities` where $\text{hp} > 0$hp > 0. If `aliveCount [op] targetSwarmSize`, routes to `trueNodeId`.

`formation_integrity`

Tests swarm geometric coherence ($0 - 100\%$0 – 100%).

- **Category**: `condition`
- **Parameters**:
  - `targetFormationIntegrity` (number, $0 - 100\%$0 – 100%): Integrity percentage. Default `80%`.
  - `formationIntegrityOp` (`'>=' | '<='`): Comparison operator. Default `>=`.
  - `trueNodeId` / `falseNodeId`: Target nodes.
- **Logic**: Compares `runtimeState.formationIntegrity` against threshold.

4. Flow Control Nodes (Green)

Flow control nodes manage iterative looping, conditional switching, multi-branching, and state transitions.

`repeat`

Loop execution node supporting fixed iteration counts or infinite repetitions.

- **Category**: `flow`
- **Icon / Visual**: `Repeat` (Green)
- **Parameters**:
  - `repeatCount` (number): Number of loop iterations. Default `3` (or `-1` for infinite).
  - `isInfiniteRepeat` (boolean): Flag for indefinite looping.
  - `trueNodeId`: Loop body branch (taken on each iteration).
  - `falseNodeId`: Loop exit branch (taken once counter exceeds `repeatCount`).
- **Runtime Logic**: Increments `repeatCounters[node.id]`. If infinite or `currentCount <= repeatCount`, routes to `trueNodeId`. Once loop limit is exceeded, resets counter to $0$0 and routes to `falseNodeId`.

`branch_switch`

Binary router directing execution based on creature aggression state.

- **Category**: `flow`

- **Icon / Visual**: `GitBranch` (Green)
- **Parameters**:
  - `trueNodeId`: High Aggression branch ($\text{aggression} \geq 60$aggression ≥ 60).
  - `falseNodeId`: Low Aggression branch ($\text{aggression} < 60$aggression < 60).
- **Runtime Logic**: Inspects `creature.aggression` and branches accordingly.

`state_transition`

Forces an immediate transition from the current HSM state to a target state.

- **Category**: `flow`
- **Icon / Visual**: `CornerDownRight` (Green)
- **Parameters**:
  - `targetStateName` (string): Name of the destination behavior state (e.g. `"Enraged"`, `"Deck Assault"`).
  - `nextNodeId` (string): Fallback node if target state name is unresolved.
- **Runtime Logic**: Finds matching state by name (case-insensitive), updates `currentStateId`, resolves destination `entryNodeId`, resets `nodeTimer = 0`, and prepends state jump to execution log.

`parallel_execute`

Triggers concurrent execution across multiple sub-branches.

- **Category**: `flow`
- **Icon / Visual**: `Layers` (Green)
- **Parameters**:
  - `nextNodeId` (string): Primary execution branch.
  - `parallelNodeIds` (string[]): Additional concurrent branch roots.
- **Runtime Logic**: Records parallel node IDs in `runtimeState.parallelActiveNodeIds` and advances along primary `nextNodeId`.

5. Special Nodes (Purple)

Special nodes handle macro-level boss phase triggers, combat event listening, and swarm synchronization.

`swarm_coordination`

Instantly locks swarm coherence and formation integrity to maximum.

- **Category**: `special`
- **Icon / Visual**: `Users` (Purple)
- **Parameters**:
  - `coherence` (number, $0 - 100\%$0 – 100%): Set coherence value. Default `95%`.
  - `nextNodeId`: Next node.
- **Runtime Logic**: Sets `runtimeState.formationIntegrity = coherence` and advances immediately.

`phase_trigger`

Advances the creature to the next combat phase (e.g. Phase 2 enrage mechanics).

- **Category**: `special`
- **Icon / Visual**: `Sparkles` (Purple)
- **Parameters**:
  - `targetPhase` (number): Specific phase integer. Defaults to `currentPhase + 1`.
  - `nextNodeId`: Next node.
- **Runtime Logic**: Updates `runtimeState.currentPhase`, logs transition, and advances.

`event_listener`

Listens for external combat triggers (such as vessel ramming or system stuns).

- **Category**: `special`
- **Icon / Visual**: `Radio` (Purple)
- **Parameters**:
  - `listenEvent` (`'HULL_RAMMED' | 'STUNNED' | 'MODULE_DESTROYED' | 'UNDER_ATTACK' | 'HP_LOW'`): Target trigger.

    - `nextNodeId`: Next node.
- **Runtime Logic**: Listens for event dispatch and advances to `nextNodeId`.

6. Swarm Kinematics & Flocking Mechanics

For swarm entities (*Veilfangs*), CombatSim combines Reynolds Boids rules with an orbital anchor pull:

1. **Separation**: Prevents crowding within `flockSeparationDist` (24px24px). Scaled inversely by distance squared.
2. **Alignment**: Normalizes headings with all flock neighbors within 120px120px.
3. **Cohesion**: Steers toward the local center of mass of neighbors within 120px120px.
4. **Orbital Roaming Anchor**: Rotates around Behemoth center at 300px300px radius, providing an anchor pull that prevents the flock from wandering off-screen while keeping full speed.

7. Apex Predator Specialized Mechanics

Tangential Orbiting & Depths Cycle (`charge_orbit`)

- Rather than cutting straight across the vessel, the creature calculates a true tangent angle: $\theta_{\text{tangent}} = \theta_{\text{orbit}} \pm 90^\circ$ θtangent = θorbit ± 90∘
- While in `depthsInterval`, the creature is submerged (`underwater = true`, `depthRatio = 1.0`), granting **100% damage immunity** from surface deck turrets.
- While in `surfaceInterval`, the creature surfaces (`underwater = false`), becoming fully targetable.

Power Grid Drain (`short_circuit_drain`)

- Latches under the keel of the Behemoth.
- Emits continuous `SHORT_CIRCUIT_DRAIN` events.
- Siphons vessel accumulator energy and rolls for system paralysis (`stunChance = 10%/sec`, `stunDuration = 10s`) against generators and defensive installations.

8. Edge Cases & Safety Fallbacks

1. **Deadlock / Unconnected Nodes**: If a designer deletes a node or leaves an output unlinked (`nextNodeId == null`), the engine triggers a safe fallback: it queries the current state's `entryNodeId`, or selects the first available node in the state, preventing creature freezing or runtime crashes.
2. **Target Building Destroyed Mid-Assault**: In `group_approach_building`, `group_strike_target`, and `flock_dissipate_regroup`, if a targeted building is destroyed while entities are en route, the engine dynamically retargets to another alive building with stationed crew members.
3. **Swarm Extinction (0 Entities Remaining)**: If all swarm entities are killed, swarm-specific loops immediately break and route to the fallback node, preventing infinite loops or divide-by-zero errors.
4. **Simulation Time Synchronization**: All timers (`nodeTimer`, `scatterTimer`, `activeNodeStartTime`) use accumulated simulation delta time dtdt, remaining deterministic regardless of frame rate fluctuations or simulation speed scaling ($1\times, 2\times, 5\times$1×, 2×, 5×).